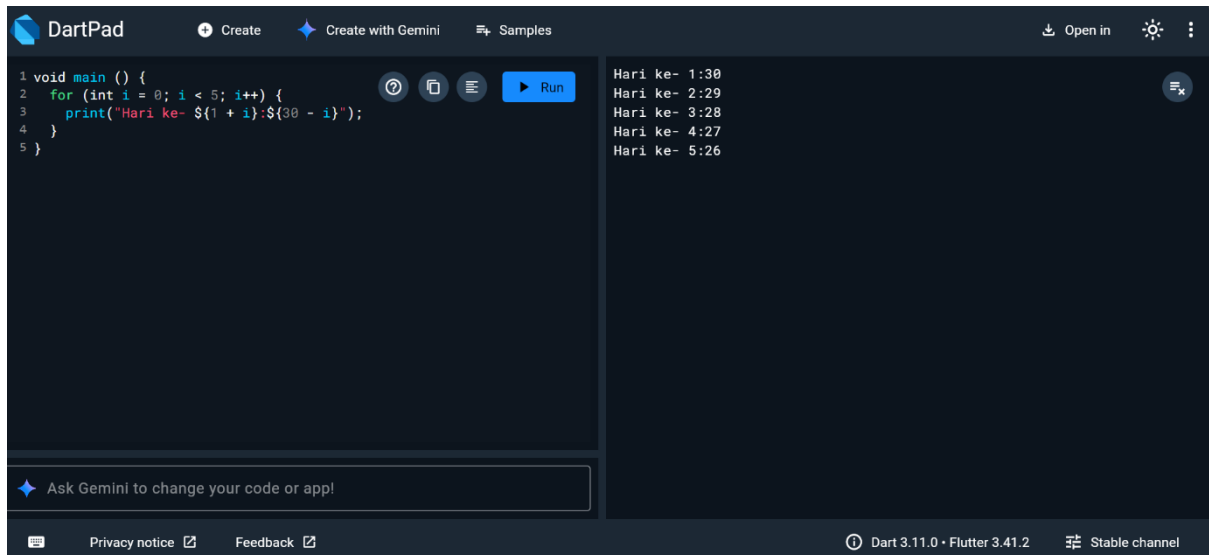


Praktikum 1 : Menerapkan Control Flows (if/else)

Tugas langkah 1 Percadangan sederhana



1. Fungsi utama

```
void main () {
```

- `main()` adalah **fungsi utama** pada program Dart.
- Program akan **mulai berjalan dari fungsi ini**.

2. Perulangan `for`

```
for (int i = 0; i < 5; i++) {
```

Perulangan ini digunakan untuk **mengulang perintah sebanyak 5 kali**.

Penjelasan bagian-bagiannya:

- `int i = 0`
→ membuat variabel `i` dengan nilai awal **0**
- `i < 5`
→ perulangan akan terus berjalan selama nilai `i` **kurang dari 5**
- `i++`
→ setiap perulangan selesai, nilai `i` **bertambah 1**

Jadi nilai `i` akan menjadi:

0, 1, 2, 3, 4

(total **5 kali perulangan**)

3. Perintah `print`

```
print("Hari ke- ${1 + i}.${30 - i}");
```

Perintah ini digunakan untuk **menampilkan teks ke layar**.

Di dalam string ada **interpolasi Dart** menggunakan `${}` yang berarti kita bisa memasukkan **perhitungan langsung ke dalam teks**.

Penjelasannya:

`${1 + i}`

Digunakan untuk menentukan **nomor hari**.

Karena `i` mulai dari 0:

```
1 + 0 = 1
1 + 1 = 2
1 + 2 = 3
1 + 3 = 4
1 + 4 = 5
```

Jadi akan muncul:

```
Hari ke-1
Hari ke-2
Hari ke-3
Hari ke-4
Hari ke-5
```

`${30 - i}`

Digunakan untuk menampilkan angka yang **berkurang setiap hari**.

```
30 - 0 = 30
30 - 1 = 29
30 - 2 = 28
30 - 3 = 27
30 - 4 = 26
```

4. Hasil output program

Program akan menampilkan:

```
Hari ke- 1:30
Hari ke- 2:29
Hari ke- 3:28
Hari ke- 4:27
Hari ke- 5:26
```

Artinya:

- **Hari bertambah**

- **Angka setelah titik dua berkurang**

Kesimpulan sederhana:

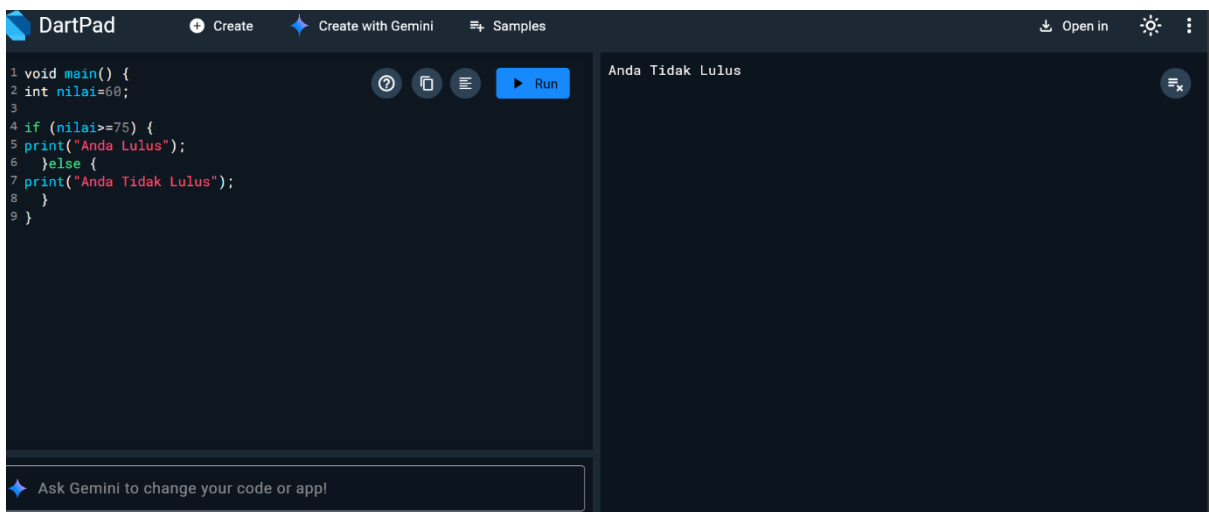
Program ini:

- Menggunakan **perulangan 5 kali**
- Menampilkan **hari yang bertambah**
- Dan **angka yang berkurang dari 30**

Kalau kamu mau, saya juga bisa jelaskan:

- **Kenapa hasilnya bisa 30, 29, 28, 27, 26**
- Atau membuat **versi flowchart dari kode ini** supaya lebih mudah dipahami.

Langkah 2 : Menggunakan if-else



```
1 void main() {
2   int nilai=60;
3
4   if (nilai>=75) {
5     print("Anda Lulus");
6   }else {
7     print("Anda Tidak Lulus");
8   }
9 }
```

Anda Tidak Lulus

Ask Gemini to change your code or app!

1. Tujuan Program

Menentukan apakah seseorang **lulus atau tidak** berdasarkan nilai yang dimiliki.

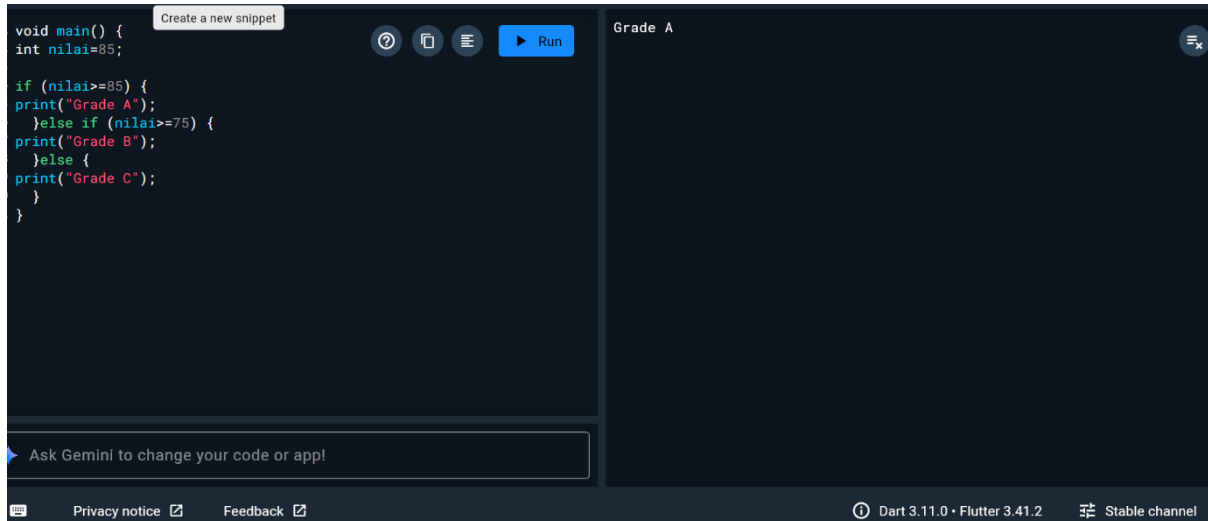
2. Alur Logika

- Program menyimpan nilai **60** ke variabel `nilai`.
- Program mengecek apakah `nilai >= 75`.
- Jika benar → tampil "**Anda Lulus**".
- Jika salah → tampil "**Anda Tidak Lulus**".
- Karena nilainya **60**, maka hasilnya "**Anda Tidak Lulus**".

3. Control Flow yang Digunakan

Program menggunakan **percabangan if-else** untuk mengambil keputusan berdasarkan kondisi nilai.

Langkah 3 : Menggunakan if-else if-else



```
void main() {  
  int nilai=85;  
  
  if (nilai>=85) {  
    print("Grade A");  
  }else if (nilai>=75) {  
    print("Grade B");  
  }else {  
    print("Grade C");  
  }  
}
```

Grade A

1. Tujuan Program

Menentukan **grade nilai (A, B, atau C)** berdasarkan nilai yang dimiliki.

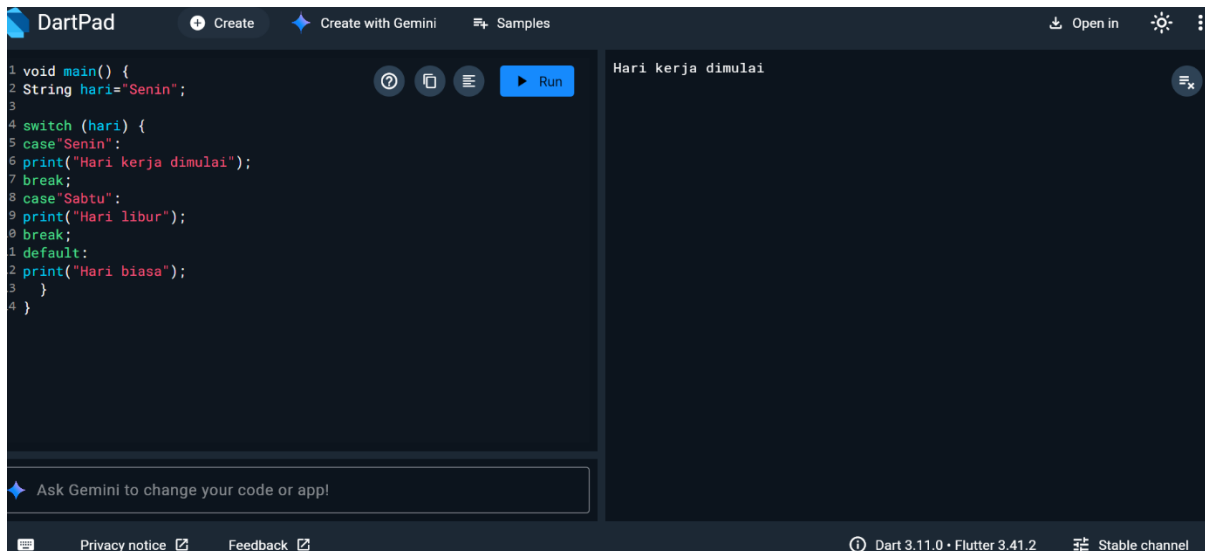
2. Alur Logika

- Program menyimpan nilai **85** ke variabel `nilai`.
- Jika `nilai >= 85` → tampil **Grade A**.
- Jika `nilai >= 75` → tampil **Grade B**.
- Jika kurang dari 75 → tampil **Grade C**.
- Karena nilainya **85**, maka hasilnya **Grade A**.

3. Control Flow yang Digunakan

Program menggunakan **percabangan if - else if - else** untuk menentukan grade berdasarkan nilai.

Langkah 4 : Switch Case Dasar



1. Tujuan Program

Tujuan program ini adalah **menentukan jenis hari berdasarkan nama hari** yang disimpan dalam variabel `hari`. Program akan menampilkan pesan apakah hari tersebut **hari kerja dimulai, hari libur, atau hari biasa**.

2. Penjelasan Alur Logika

1. Program dimulai dari fungsi `main()`.
2. Dibuat sebuah variabel `String` bernama `hari` dengan nilai "Senin".
3. Program kemudian menggunakan **switch** untuk mengecek nilai dari variabel `hari`.
4. Jika nilai `hari` adalah:
 - o "Senin" → program menampilkan "**Hari kerja dimulai**".
 - o "Sabtu" → program menampilkan "**Hari libur**".
5. Jika nilai `hari` tidak cocok dengan pilihan yang ada, maka bagian `default` akan dijalankan dan menampilkan "**Hari biasa**".

Karena pada program nilai `hari` adalah "Senin", maka output yang muncul adalah:

```
Hari kerja dimulai
```

3. Penjelasan Control Flow yang Digunakan

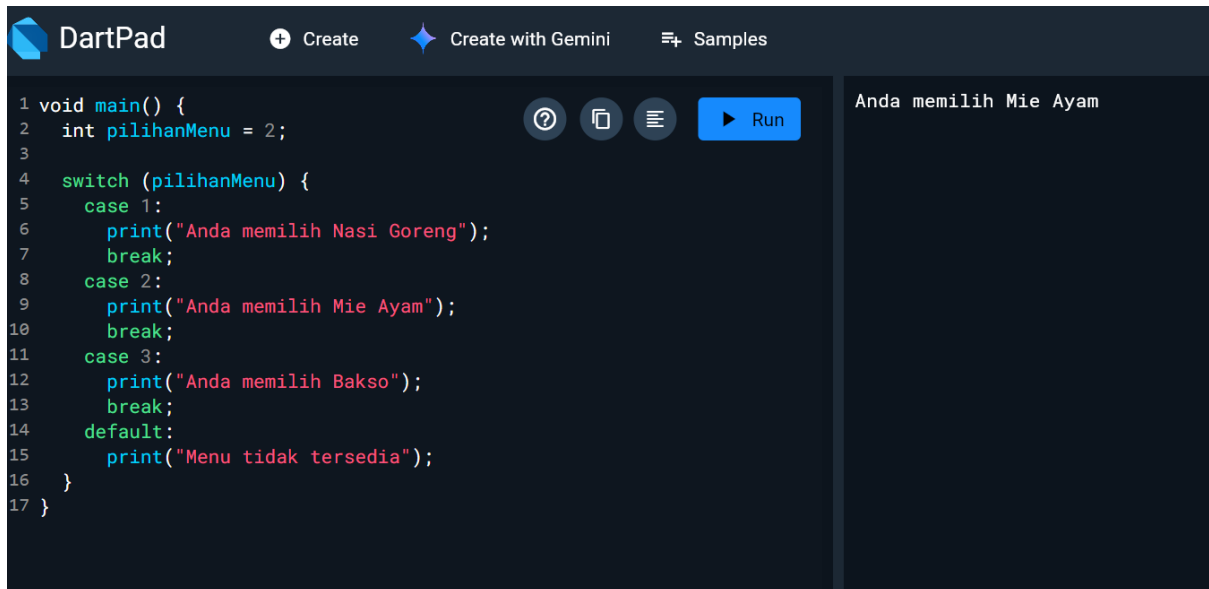
Program ini menggunakan **control flow switch-case**.

Fungsi `switch` adalah untuk **memilih satu kondisi dari beberapa pilihan berdasarkan nilai variabel**.

- `case` → kondisi yang dicek.
- `break` → menghentikan pengecekan setelah kondisi terpenuhi agar tidak lanjut ke `case` berikutnya.
- `default` → dijalankan jika tidak ada `case` yang cocok.

Jadi, **switch-case** digunakan untuk membuat percabangan program berdasarkan nilai variabel **hari**.

Langkah 5 : Switch Case dengan Angka



```
1 void main() {
2   int pilihanMenu = 2;
3
4   switch (pilihanMenu) {
5     case 1:
6       print("Anda memilih Nasi Goreng");
7       break;
8     case 2:
9       print("Anda memilih Mie Ayam");
10      break;
11     case 3:
12       print("Anda memilih Bakso");
13       break;
14     default:
15       print("Menu tidak tersedia");
16   }
17 }
```

Anda memilih Mie Ayam

1. Tujuan Program

Tujuan program ini adalah **menentukan menu makanan yang dipilih berdasarkan nomor menu**. Program akan menampilkan nama makanan sesuai dengan angka yang tersimpan pada variabel **pilihanMenu**.

2. Penjelasan Alur Logika

1. Program dimulai dari fungsi `main()`.
2. Dibuat variabel bertipe `int` bernama `pilihanMenu` dengan nilai **2**.
3. Program menggunakan **switch** untuk mengecek nilai dari `pilihanMenu`.
4. Jika nilai yang cocok ditemukan:
 - o case 1 → menampilkan "**Anda memilih Nasi Goreng**"
 - o case 2 → menampilkan "**Anda memilih Mie Ayam**"
 - o case 3 → menampilkan "**Anda memilih Bakso**"
5. Jika angka yang dimasukkan tidak ada dalam pilihan, maka **default** akan dijalankan dan menampilkan "**Menu tidak tersedia**".

Karena nilai `pilihanMenu = 2`, maka output yang muncul adalah:

Anda memilih Mie Ayam

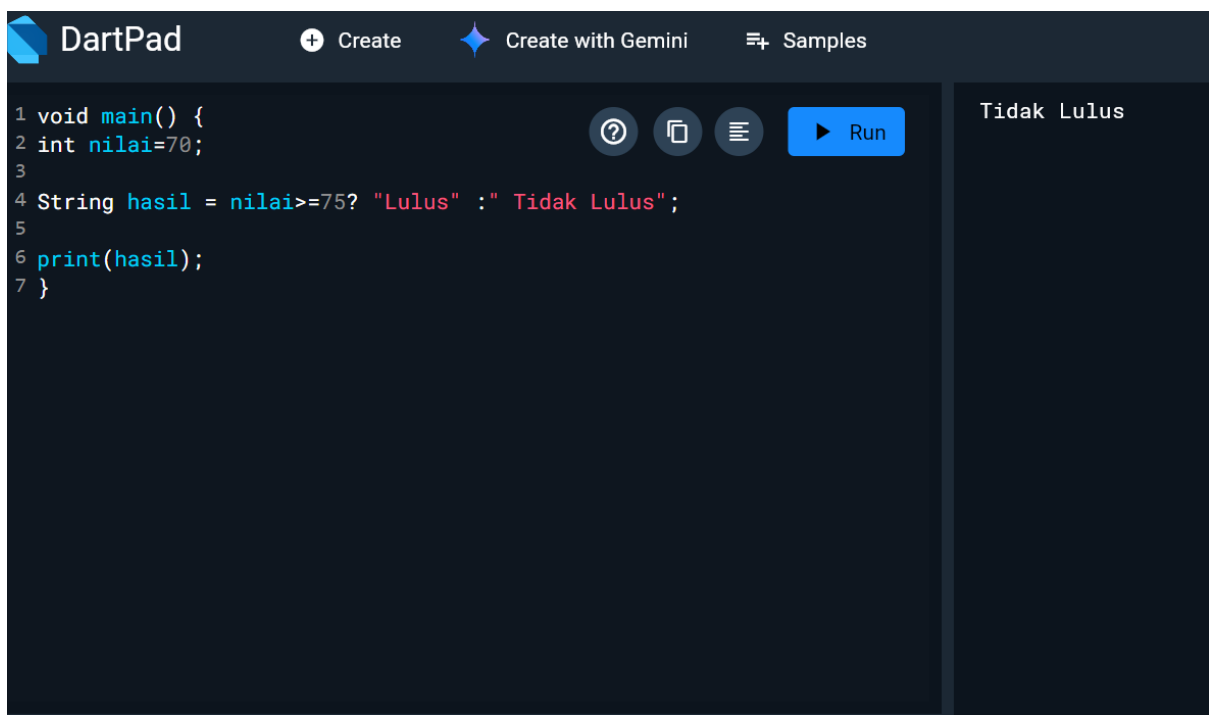
3. Penjelasan Control Flow yang Digunakan

Program ini menggunakan **control flow switch-case**.

- `switch` digunakan untuk **memeriksa nilai variabel dan memilih salah satu kondisi yang sesuai**.
- `case` berisi kemungkinan nilai yang dicek.
- `break` digunakan untuk **menghentikan proses setelah kondisi terpenuhi agar tidak lanjut ke case berikutnya**.
- `default` dijalankan jika **tidak ada case yang cocok**.

Jadi, **switch-case membantu program memilih menu makanan berdasarkan nomor yang dipilih**.

Langkah 6 : Menggunakan Operator Ternary



```

1 void main() {
2   int nilai=70;
3
4   String hasil = nilai>=75? "Lulus" : "Tidak Lulus";
5
6   print(hasil);
7 }
  
```

Tidak Lulus

1. Tujuan Program

Tujuan program ini adalah **menentukan apakah seorang siswa lulus atau tidak berdasarkan nilai yang diperoleh**.

2. Penjelasan Alur Logika

1. Program dimulai dari fungsi `main()`.
2. Dibuat variabel `nilai` bertipe `int` dengan nilai **70**.
3. Program menggunakan **operator ternary** (`? :`) untuk mengecek apakah nilai tersebut **lebih besar atau sama dengan 75**.
4. Jika kondisi `nilai >= 75` benar, maka variabel `hasil` akan berisi **"Lulus"**.
5. Jika kondisi salah, maka variabel `hasil` akan berisi **"Tidak Lulus"**.
6. Nilai dari variabel `hasil` kemudian ditampilkan menggunakan `print()`.

Karena nilai yang diberikan adalah **70**, yang **lebih kecil dari 75**, maka output yang muncul adalah:

Tidak Lulus

3. Penjelasan Control Flow yang Digunakan

Program ini menggunakan **control flow berupa operator ternary** (`?:`).

- Operator ternary adalah **bentuk singkat dari percabangan if-else**.
- Struktur penulisannya:

```
kondisi ? nilai_jika_benar : nilai_jika_salah
```

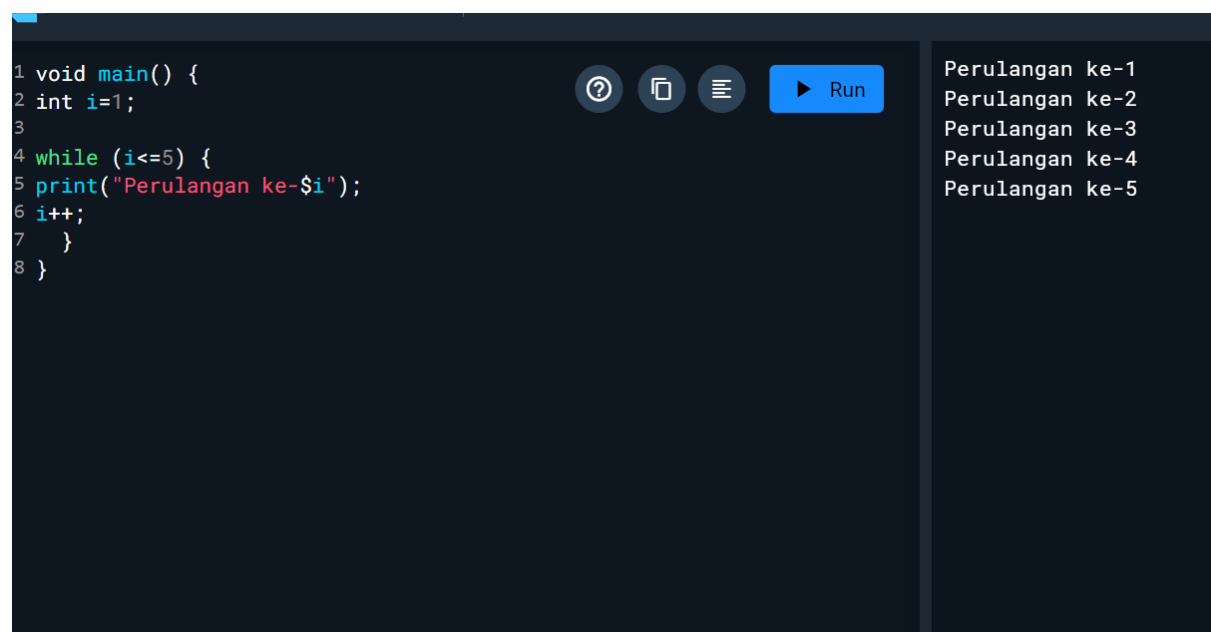
Pada program ini:

- Jika `nilai >= 75` → hasil **"Lulus"**
- Jika tidak → hasil **"Tidak Lulus"**

Jadi, **operator ternary digunakan untuk menentukan status kelulusan berdasarkan nilai dengan cara yang lebih singkat dibandingkan if-else**.

Paktikum 2 : Menerapkan Perulangan “while” dan “do-while”

Langkah 1: perulangan while sederhana



```
1 void main() {
2   int i=1;
3
4   while (i<=5) {
5     print("Perulangan ke-$i");
6     i++;
7   }
8 }
```

Perulangan ke-1
Perulangan ke-2
Perulangan ke-3
Perulangan ke-4
Perulangan ke-5

1. Tujuan Program

Program ini bertujuan untuk **menampilkan teks perulangan dari 1 sampai 5**.

2. Penjelasan Alur Logika

Program dimulai dengan variabel `i = 1`. Kemudian program menggunakan **while** untuk melakukan perulangan selama nilai `i` **kurang dari atau sama dengan 5**. Setiap perulangan akan menampilkan tulisan "**Perulangan ke-i**" lalu nilai `i` bertambah satu (`i++`) sampai mencapai 5.

3. Penjelasan Control Flow / Perulangan

Program ini menggunakan **perulangan while**, yaitu perulangan yang berjalan selama kondisi yang diberikan bernilai **true**. Ketika nilai `i` sudah lebih dari 5, perulangan akan berhenti.

Langkah 2: While dengan kondisi logika

```
1 void main() {
2   int nilai=50;
3
4   while (nilai<75) {
5     print("Nilai sekarang:$nilai");
6     nilai+=5;
7   }
8
9   print("Nilai memenuhi syarat");
10 }
```

Run

Nilai sekarang:50
Nilai sekarang:55
Nilai sekarang:60
Nilai sekarang:65
Nilai sekarang:70
Nilai memenuhi syarat

1. Tujuan Program

Program ini bertujuan untuk **meningkatkan nilai secara bertahap sampai mendekati nilai 75** dan menampilkan setiap perubahan nilai.

2. Penjelasan Alur Logika

Program dimulai dengan variabel `nilai = 50`. Kemudian program menggunakan **while** untuk melakukan perulangan selama nilai masih **kurang dari 75**. Setiap perulangan program menampilkan nilai saat ini lalu nilai ditambah 5 (`nilai += 5`). Setelah kondisi tidak terpenuhi, program menampilkan "**Nilai memenuhi syarat**".

3. Penjelasan Control Flow / Perulangan

Program ini menggunakan **perulangan while**, yaitu perulangan yang berjalan selama kondisi `nilai < 75` masih benar. Ketika nilai sudah tidak memenuhi kondisi tersebut, perulangan berhenti.

Langkah 3: Do-While dasar

```
void main() {
    int i=1;

    do {
        print("Perulangan ke-$i");
        i++;
    }while (i<=5);
}
```

Perulangan ke-1
Perulangan ke-2
Perulangan ke-3
Perulangan ke-4
Perulangan ke-5

Struktur Utama

- **Inisialisasi** (`int i = 1;`): Menetapkan variabel penghitung (counter) dimulai dari angka 1.
- **Blok do { ... }**: Bagian kode yang akan dijalankan terlebih dahulu sebelum mengecek kondisi.
- **Kondisi while** (`i <= 5`)`;`: Syarat yang harus terpenuhi agar perulangan terus berlanjut.

Alur Logika (Langkah demi Langkah)

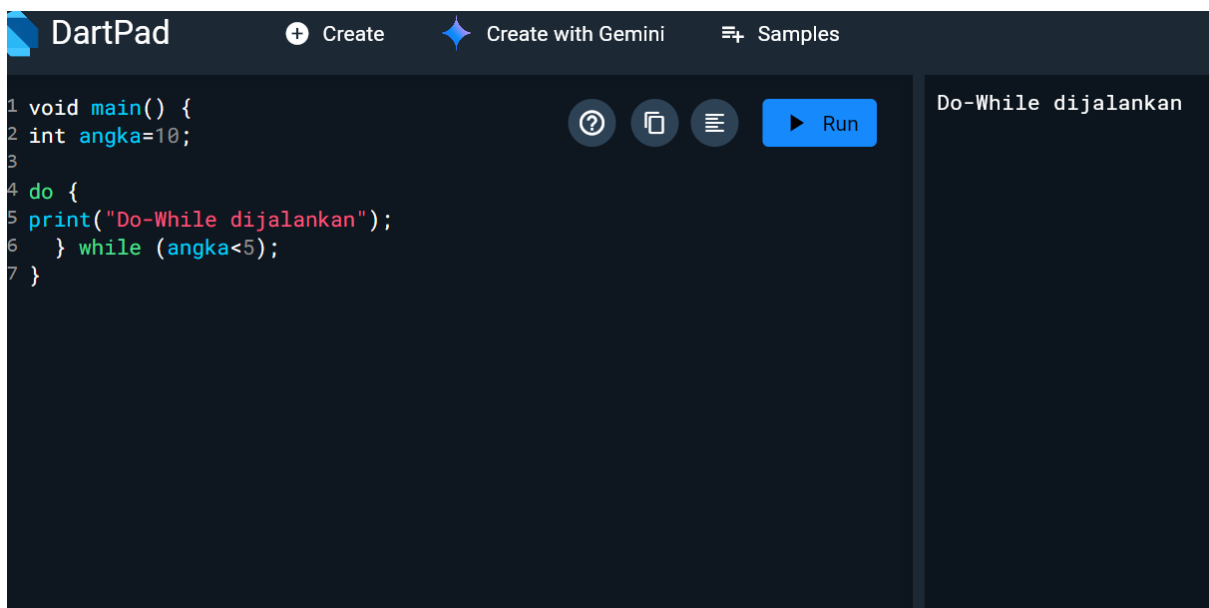
1. **Eksekusi Pertama**: Program langsung masuk ke dalam blok `do`. Ia mencetak "**Perulangan ke-1**", lalu menambah nilai `i` menjadi 2 (`i++`).
2. **Cek Kondisi**: Program mengecek apakah `2 <= 5`? Jawabannya **Ya (True)**, maka perulangan berlanjut.
3. **Iterasi Berulang**: Proses cetak dan penambahan nilai `i` terus dilakukan selama hasil pengecekan kondisi masih bernilai **True**.
 - Cetak "Perulangan ke-2", `i` jadi 3. (`3 <= 5`? True)
 - Cetak "Perulangan ke-3", `i` jadi 4. (`4 <= 5`? True)
 - Cetak "Perulangan ke-4", `i` jadi 5. (`5 <= 5`? True)
 - Cetak "Perulangan ke-5", `i` jadi 6. (`6 <= 5`? **False**)

4. **Berhenti:** Karena pada pengecekan terakhir nilai `i` sudah menjadi **6**, kondisi `6 <= 5` bernilai **False**, sehingga perulangan berhenti.

Poin Penting

Karakteristik utama dari `do-while` adalah **minimal satu kali eksekusi**. Artinya, meskipun seandainya kamu mengatur `i = 10` di awal, program akan tetap mencetak "Perulangan ke-10" satu kali sebelum menyadari bahwa kondisinya salah.

Langkah 4 : Perbedaan while dan Do-while



The screenshot shows the DartPad interface. On the left, a code editor contains the following Dart code:

```
1 void main() {  
2   int angka=10;  
3  
4   do {  
5     print("Do-While dijalankan");  
6   } while (angka<5);  
7 }
```

On the right, a console output area displays the text "Do-While dijalankan". Above the console, there are icons for help, copy, and a "Run" button.

Mengapa Teks Tetap Muncul?

Meskipun variabel `angka = 10` dan syarat perulangannya adalah `angka < 5` (yang secara logika bernilai **salah/false**), teks "Do-While dijalankan" tetap muncul di konsol.

Alur Logikanya

1. **Eksekusi Tanpa Syarat:** Program masuk ke blok `do` terlebih dahulu. Tanpa melihat kondisi apa pun, ia langsung menjalankan perintah `print("Do-While dijalankan");`.
2. **Pengecekan di Akhir:** Setelah mencetak teks, barulah program sampai di baris `while (angka < 5);`.
3. **Evaluasi:** Program mengecek: "Apakah 10 lebih kecil dari 5?". Jawabannya adalah **False**.

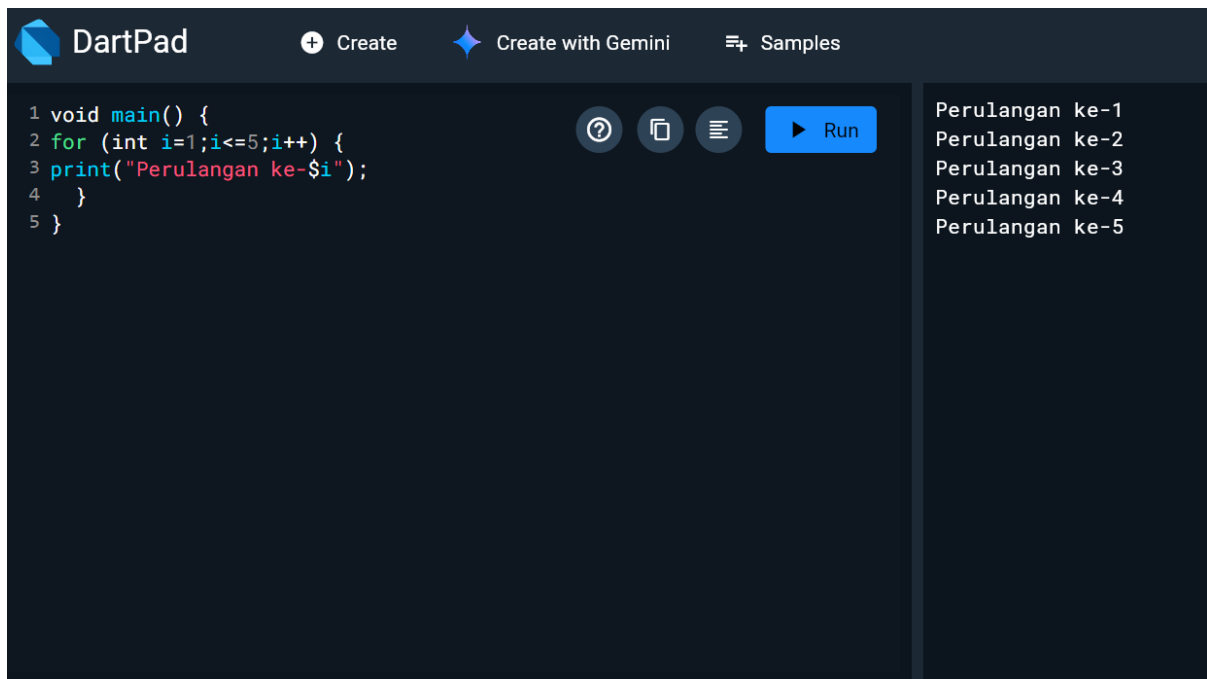
4. **Berhenti:** Karena hasilnya False, perulangan berhenti saat itu juga dan tidak mengulang ke atas lagi.

Kesimpulan

Perulangan `do-while` menjamin bahwa kode di dalam bloknya akan **dieksekusi minimal satu kali**, tidak peduli apakah kondisinya benar atau salah sejak awal. Ini sangat berguna jika kamu ingin menampilkan menu atau meminta input user setidaknya satu kali sebelum divalidasi.

Praktikum 3: Menerapkan perulangan “for” dan “break-continue”

Langkah 1: Perulangan for Dasar



The screenshot shows the DartPad interface. On the left, a code editor contains the following Dart code:

```
1 void main() {  
2   for (int i=1;i<=5;i++) {  
3     print("Perulangan ke-$i");  
4   }  
5 }
```

On the right, the output console displays the results of the program execution:

```
Perulangan ke-1  
Perulangan ke-2  
Perulangan ke-3  
Perulangan ke-4  
Perulangan ke-5
```

1. Tujuan Program

Program ini bertujuan untuk **menampilkan tulisan “Perulangan ke-1” sampai “Perulangan ke-5” di layar** menggunakan perulangan.

2. Penjelasan Alur Logika

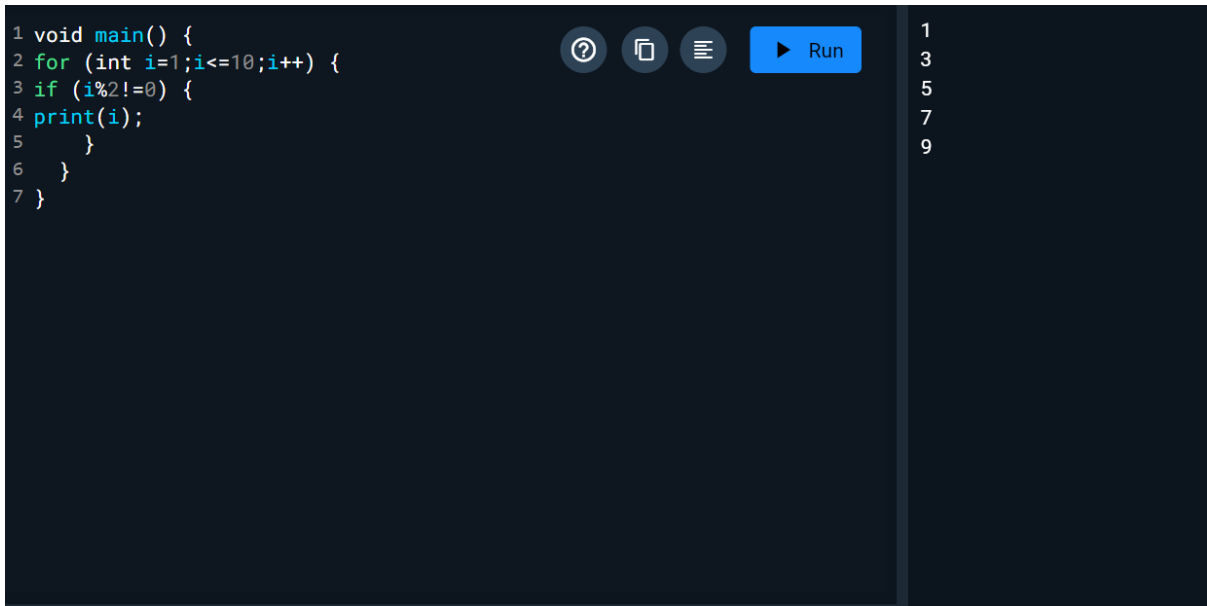
Program dimulai dari fungsi `main()`. Kemudian variabel `i` dimulai dari nilai **1**. Program akan mencetak tulisan **“Perulangan ke-\$i”**. Setelah itu nilai `i` bertambah **1** setiap perulangan sampai mencapai **5**. Jika nilai `i` sudah lebih dari **5**, maka perulangan berhenti.

3. Penjelasan Control Flow / Perulangan

Program menggunakan **perulangan for**. Perulangan `for` digunakan untuk menjalankan suatu perintah **berulang kali dengan jumlah yang sudah ditentukan**, yaitu dari **1 sampai 5**.

Langkah 2: Menampilkan bilangan ganjil

```
1 void main() {
2   for (int i=1;i<=10;i++) {
3     if (i%2!=0) {
4       print(i);
5     }
6   }
7 }
```



The screenshot shows a code editor with a dark theme. On the left, there is C++ code that defines a `main` function. Inside, a `for` loop iterates from `i=1` to `i=10`. Within the loop, an `if` statement checks if `i` is odd (`i%2 != 0`). If true, it calls `print(i)`. On the right side of the editor, the output of the program is displayed as a vertical list of numbers: 1, 3, 5, 7, and 9. Above the code editor, there are icons for help, copy, and a 'Run' button.

1. Tujuan Program

Program ini bertujuan untuk **menampilkan angka ganjil dari 1 sampai 10** di layar.

2. Penjelasan Alur Logika

Program dimulai dari fungsi `main()`.

Kemudian perulangan `for` dimulai dari **`i = 1`** sampai **`i ≤ 10`**.

Setiap perulangan, program akan mengecek apakah **`i` adalah bilangan ganjil** dengan kondisi `i % 2 != 0`.

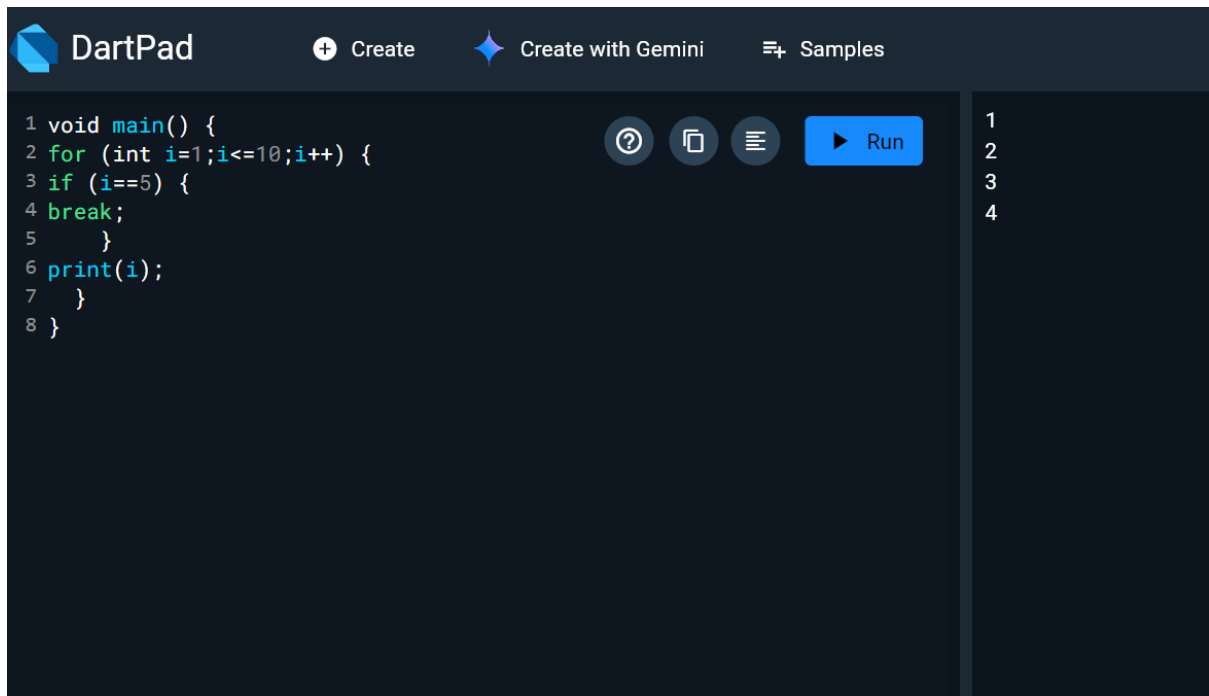
Jika kondisi benar, maka angka **`i`** akan dicetak. Jika tidak, angka tersebut dilewati.

3. Penjelasan Control Flow / Perulangan

Program menggunakan **perulangan `for`** untuk mengulang dari angka **1 sampai 10**.

Di dalamnya terdapat **percabangan `if`** yang berfungsi untuk **menyeleksi hanya angka ganjil** yang akan ditampilkan.

Langkah 3 : Menghentikan perulangan

The image shows a screenshot of the DartPad web interface. At the top, there is a dark blue header with the DartPad logo on the left, and three buttons: 'Create' (with a plus icon), 'Create with Gemini' (with a star icon), and 'Samples' (with a list icon). Below the header is a code editor with a dark background. The code is written in Dart and is as follows:

```
1 void main() {  
2   for (int i=1;i<=10;i++) {  
3     if (i==5) {  
4       break;  
5     }  
6     print(i);  
7   }  
8 }
```

To the right of the code editor, there are three circular icons: a question mark, a copy icon, and a menu icon. Further right is a blue button with a play icon and the text 'Run'. To the right of the code editor is a vertical panel showing the output of the program, which consists of the numbers 1, 2, 3, and 4, each on a new line.

1. Tujuan Program

Program ini bertujuan untuk **menampilkan angka dari 1 sampai sebelum 5**, karena perulangan akan berhenti saat angka 5.

2. Penjelasan Alur Logika

Program dimulai dari fungsi `main()`.

Perulangan `for` dimulai dari **`i = 1`** sampai **`i ≤ 10`**.

Setiap perulangan, program mengecek apakah **`i == 5`**.

Jika nilai **`i = 5`**, maka perintah `break` dijalankan sehingga **perulangan langsung berhenti**.

Jika belum 5, maka angka **`i`** akan ditampilkan.

Output yang muncul:

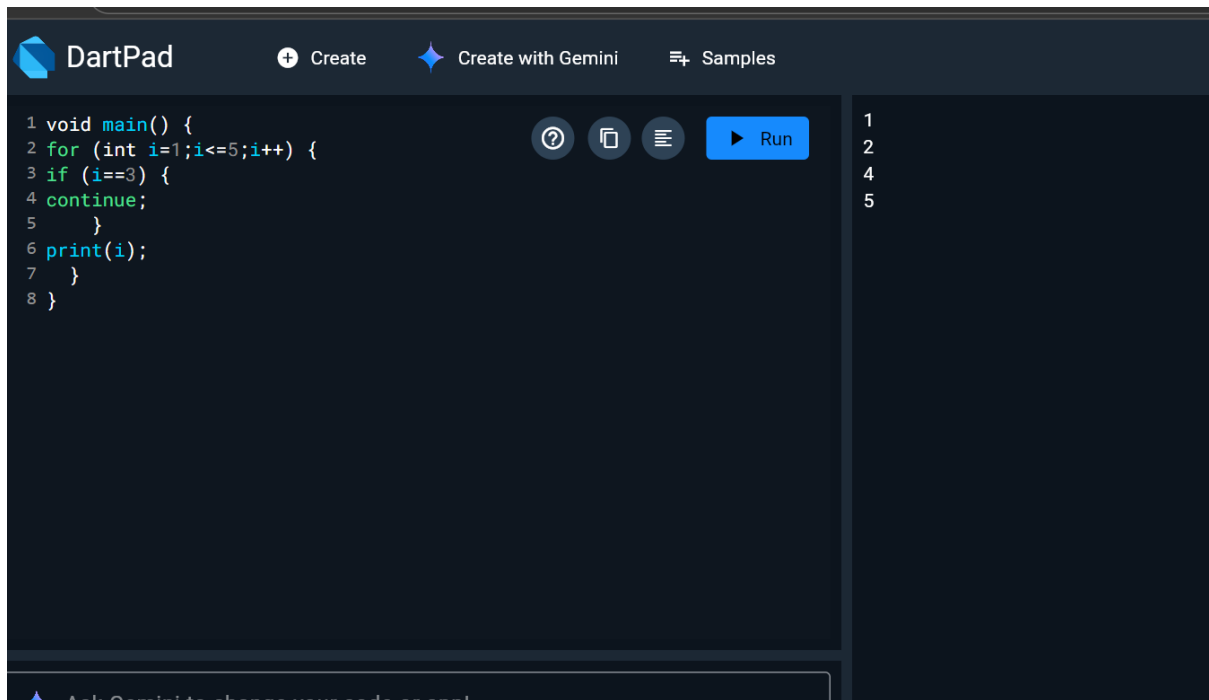
```
1  
2  
3  
4
```

3. Penjelasan Control Flow / Perulangan

Program menggunakan **perulangan `for`** untuk menghitung dari 1 sampai 10.

Di dalamnya ada **percabangan `if`** dan **perintah `break`** yang berfungsi untuk **menghentikan perulangan lebih awal ketika kondisi terpenuhi**.

Langkah 4: Melewati angka tertentu



The screenshot shows the DartPad web interface. The top navigation bar includes the DartPad logo, a 'Create' button, a 'Create with Gemini' button, and a 'Samples' button. The main editor area contains the following Dart code:

```
1 void main() {  
2   for (int i=1;i<=5;i++) {  
3     if (i==3) {  
4       continue;  
5     }  
6     print(i);  
7   }  
8 }
```

To the right of the code editor is a toolbar with icons for help, copy, and a 'Run' button. Below the code editor is a text input field with the placeholder text 'Ask Gemini to change your code or app!'. On the far right, a vertical panel displays the output of the code, showing the numbers 1, 2, 4, and 5, each on a new line.